

Reviewer Pack — Minimal Number of Projective Plane Points and Communication ...

Entry 67e0b5336b97 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 17:57:34 UTC

Minimal Number of Projective Plane Points and Communication Complexity Rank

Entry ID: 67e0b5336b97 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 17:57:34 UTC

1. Conjecture

Field A (mathematical branch): Projective Geometry (Projective Plane Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every d -regular graph G , the minimal number of points in the projective plane that can be used to represent G is linearly correlated with its communication complexity rank $r(G)$, such that the number of points is $\Theta(r(G))$.

Rationale (proposer's reasoning):

Projective geometry provides a way to embed graphs into geometric spaces, which may reveal hidden structures or symmetries in the graph's communication complexity. The projective plane has been used in combinatorial problems and could potentially offer new insights into communication complexity.

Taxonomy category: projective-geometry-to-communication-complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cae9a69a2f24a0a8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each seed, the ratio of the number of points in the projective plane to the communication complexity rank must fall within $[0.5, 1.5]$ for at least 80% of the graphs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "projective geometry" AND "communication complexity" AND "matrix rank"
- "d-regular graph" AND "projective plane" AND "communication complexity rank" - "minimal number of points" AND "projective geometry" AND θ ("communication complexity rank")

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d:
                    if (i, j) not in edges and (j, i) not in edges:
                        graph[i].append(j)
                        graph[j].append(i)
                        edges.add((i, j))
        return graph

    def projective_plane_points(graph):
        n = len(graph)
        points = set()
        for i in range(n):
            for j in range(i + 1, n):
                if (i, j) in graph:
                    point = (i, j)
                    points.add(point)
        return len(points)

    def communication_complexity_rank(graph):
        n = len(graph)
        rank = 0
```

```

    for i in range(n):
        neighbors = set(graph[i])
        for j in range(i + 1, n):
            if j not in neighbors:
                common_neighbors = set(graph[j]) & neighbors
                rank += len(common_neighbors)
    return rank

def run_test(n, d):
    graph = generate_d_regular_graph(n, d)
    if graph is None:
        return {"metric_name": "points_to_rank_ratio", "metric_value": float('inf'), "instances_tested": 0}

    points = projective_plane_points(graph)
    rank = communication_complexity_rank(graph)
    ratio = points / rank if rank != 0 else float('inf')
    return {"metric_name": "points_to_rank_ratio", "metric_value": ratio, "instances_tested": 1}

results = []
for d in [2, 3, 4]:
    for _ in range(10):
        result = run_test(random.randint(5, 40), d)
        results.append(result)

total_points = sum(r["metric_value"] * r["instances_tested"] for r in results)
total_rank = sum(r["instances_tested"] for r in results)
mean_ratio = total_points / total_rank if total_rank != 0 else float('inf')

conjecture_holds = all(0.5 <= r["metric_value"] <= 1.5 for r in results)
counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), "")

return {"metric_name": "points_to_rank_ratio", "metric_value": mean_ratio, "instances_tested": len(results)}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] * r["instances_tested"] for r in results) / sum(r["instances_tested"] for r in results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_ratio)**2 * r["instances_tested"] for r in results) / sum(r["instances_tested"] for r in results))
    support_fraction = sum(1 for r in results if 0.5 <= r["metric_value"] <= 1.5) / len(results)

    if all(0.5 <= r["metric_value"] <= 1.5 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[seeds.index(counterexample)]}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
example': 'graph_not_possible'}
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'points_to_rank_ratio', 'metric_value': inf, 'instances_tested': 30, 'n_max':
RESULT: FALSIFIED counterexample="graph_not_possible" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only considers a very small range of n (5 to 40) and d (2, 3, 4), which is insufficient to validate the conjecture over all possible regular graphs. The metric may not scale trivially with n , and the results could be due to chance or specific properties of the tested graphs.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one seed, as the ratio of the number of points in the projective plane to the | next: Expand the range of n and d to include a broader spectrum of regular graphs and retest the conjecture. Additionally, consider using more sophisticated methods to validate the conjecture over a larger set of graphs.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11374
2	propose	ollama_remote	glm4:latest	0	0	13356
3	preregistration	ollama_remote	glm4:latest	0	0	10162
4	novelty	ollama_remote	glm4:latest	0	0	9517
5	novelty	ollama_remote	glm4:latest	0	0	11736
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15248
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11397
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11918
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11934

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	critic	ollama_remote	glm4:latest	0	0	17142
11	judge	ollama_remote	glm4:latest	0	0	9660

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133446 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/67e0b5336b97.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/67e0b5336b97.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/67e0b5336b97.tar.gz (if generated)